

Sistemi informativi su Web

Gestione Persistenza: JPQL

aa 2023-2024



Paolo Merialdo
Università degli Studi Roma Tre



This work is licensed under a Creative Commons Attribution-Noncommercial-Share Alike 3.0 United States
See <http://creativecommons.org/licenses/by-nc-sa/3.0/us/> for details

JPQL

- JPA, oltre alla possibilità di specificare il mapping object-relational, offre un linguaggio di interrogazione: JPQL
- JPQL permette di interrogare l'insieme di entità memorizzate nel db
- JPQL è simile all'SQL, ma ha una sintassi più vicina al mondo OO
- Istruzioni JPQL
 - SELECT
 - UPDATE
 - DELETE

JPQL: SELECT

SELECT <espressione select>

FROM <clausola from>

[WHERE <espressione condizionale>]

[ORDER BY <clausola order by>]

[GROUP BY <clausola group by>]

[HAVING <clausola having>]

JPQL: SELECT

- Un'istruzione SELECT tipicamente restituisce un'entità o una lista di entità
- La clausola FROM definisce l'entità coinvolta attraverso una variabile di identificazione

SELECT c

FROM Customer c

Variabile di
identificazione

Entità
deve essere scritta come la classe:
con l'iniziale maiuscola

JPQL: SELECT

- La clausola WHERE permette di specificare condizioni per restringere il risultato di una SELECT alle istanze che soddisfano la condizione

```
SELECT c  
FROM Customer c  
WHERE c.firstName='Antonio'
```

JPQL: SELECT

- La clausola WHERE permette di specificare condizioni che permettono di inseguire i riferimenti: si usa una *path-expression* con la tipica *dot-notation* OO
 - questo elimina la necessità di usare join: lavoriamo su entità e non su tabelle
 - JPA traduce le query JPQL in query SQL che invia al DBMS. I join vengono quindi effettuati, ma sintatticamente li esprimiamo attraverso la navigazione dei riferimenti

```
SELECT c  
FROM Customer c  
WHERE c.firstName='Antonio' AND c.address.city='Roma'
```

Questo si tradurrà in un join tra la tabella
Customer e la tabella Address

JPQL: DELETE

- Sintassi:

```
DELETE FROM <nome entità> [[AS] alias]  
[WHERE <espressione condizionale>]
```

* Le operazioni di delete su un **insieme di istanze** vanno fatte **sempre** con una query JPQL. Eliminare un insieme di istanze una per una (ad esempio con un ciclo) è **molto inefficiente**

JPQL: DELETE

- Esempio:

```
DELETE FROM Customer c
```

```
WHERE c.address.city = 'Genova'
```

JPQL: UPDATE

- Analogamente è possibile fare aggiornamenti con l'istruzione JPQL UPDATE
- Sintassi:
UPDATE <nome entità> [[AS] alias]
SET <update> {,<update>}
[WHERE <espressione condizionale>]
- Esempio:
UPDATE Product p
SET p.price = p.price - 1
WHERE p.price > 30

Anche in questo caso, operazioni di update su un **insieme di istanze** vanno fatte **sempre** con una query JPQL. Aggiornare un insieme di istanze una per una è **molto inefficiente**

JPQL: Tipologie di query

- Dynamic Queries: specificate runtime
- Named Queries: statiche e immutabili, sono definite nella entità (vedi prox slide)
- Native Queries: query SQL

Named Queries

- Sono definite in maniera statica e immutabile nella entità a cui fanno riferimento
 - Prima della definizione della classe
 - Tramite l'annotazione
`@NamedQuery("nome", query="jpql query")`
- Possono essere sia query di interrogazione che query di aggiornamento (update o delete)
- Possono essere parametriche

Named Queries: esempio

```
package it.uniroma3.siw.model;
```

```
import javax.persistence.Entity; [...]
```

```
@Entity
```

```
@NamedQuery(name = "Product.findAllProducts", query = "SELECT p FROM Product p")
```

```
@NamedQuery(name = "Product.deleteAllProducts", query = "DELETE Product p")
```

```
public class Product {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.AUTO)
```

```
    private Long id;
```

```
    ...
```

```
    ....
```

JPQL: query parametriche

- JPQL permette di definire le query in modo parametrico, con il valore dei parametri definito dinamicamente
 - Supponiamo ad esempio di dover definire un metodo che restituisce la lista dei clienti di una città, con la città passata come parametro. Dovremmo scrivere una query in cui il valore della città è parametrico

JPQL: query parametriche

- I parametri vanno indicati nella query e successivamente istanziati
- In una JPQL query (named o dynamic) i parametri vanno indicati con un identificatore preceduto dal simbolo :
- Esempio:
`SELECT c FROM Customer c WHERE c.firstName = :name`

JPQL: group by, having, order by

- Le query con aggregazioni hanno una sintassi ed una semantica equivalente a quelle in SQL
- Attenzione: se la nostra applicazione prevede operazioni che aggregano dati persistenti, bisogna far eseguire le aggregazioni dal DBMS. Sarebbe molto inefficiente (e se i dati sono tanti, impossibile) caricare (ad esempio con una `findAll()`) tutti gli oggetti in memoria e effettuare (in Java) le operazioni di aggregazione

JPQL: SELECT

SELECT <espressione select>

FROM <clausola from>

[WHERE <espressione condizionale>]

[ORDER BY <clausola order by>]

[GROUP BY <clausola group by>]

[HAVING <clausola having>]

Esempio

- Supponiamo che la nostra applicazione abbia bisogno di una mappa costruita con i dati dei clienti (istanze della entità Customer) nel database. La mappa associa un nome al numero di clienti con quel nome:

— Es. Paolo 5

Anna 4

Luigi 8

```
SELECT c.firstName, count(c)
FROM Customer c
GROUP BY c.firstName
```

JPQL: group by, having, order by

- Il tipo di ritorno di una SELECT con GROUP BY è una lista di array di Object
 - `List<Object[]>`
- Ogni elemento della lista offre in un array la ennupla risultato della query
- Nel nostro esempio, il primo elemento è una string con il nome del Customer, il secondo elemento è un Integer con il numero delle persone con quel nome